

EQUIPO 4
ELIXIR

PROGRAMACION LOGICA Y FUNCIONAL - 4 PM

1

ESPAÑA PEREZ MIGUEL ANGEL

2

ESTRADA RODRIGUEZ MELANI

3

FUENTES MONTAÑO AXEL

4

GARCIA CARO CARLOS ALEJANDRO

¿QUÉ ES ELIXIR?

Elixir es un lenguaje funcional creado por José Valim que se ejecuta sobre Beam, la maquina virtual de Erlang



A. Conserva concurrencia, distribución y tolerancia a fallos de Erlang.

B. Añade una sintaxis más accesible y herramientas modernas

C. Puede utilizar directamente las bibliotecas de Erlang/OTP

C. Está orientado a aplicaciones escalables, mantenibles y confiables.

DE EXPERIMENTO A LENGUAJE MODERNO



José Valim combinó la madurez de BEAM con un lenguaje funcional pensado para el trabajo cotidiano.

MODELO DE CÓMPUTO:
BEAM + ACTOR MODEL

A

Procesos ligeros (no hilos del SO):

millones de procesos aislados, memoria propia, comunicación solo por mensajes asíncronos.

B

Scheduler preemptive por núcleo:

soft real-time y baja latencia.

C

Supervisión y “let it crash”:

árboles de supervisión OTP.

D

GenServer:

la abstracción estándar para procesos con estado (callback-based). Ejemplo mental: un GenServer que representa una sesión de usuario o un “Guild” (servidor de Discord).

SINTAXIS
QUE HACE EL MODELO USABLE

Elemento	Qué aporta	Ejemplo clave
Operador pipe |>	Composición legible de transformaciones	data |> transform |> filter |> map
Stream	Evaluación perezosa (lazy) de colecciones	Procesar millones de eventos sin materializar todo en memoria
Macros	Metaprogramación higiénica (código como datos)	Extender el lenguaje, DSLs, generar GenServers, etc.

SISTEMA DE TIPOS / RUNTIME

1 Antes de correr (Compilación)

Claves de Structs: Valida campos existentes en desarrollo.

Typespecs (`@spec`): Análisis estático opcional con Dialyzer.

```
# Error en compilación (elixirc)
%Usuario{nombre: "Ana", correo: "a@m.com"}
➔ (CompileError) unknown key :correo
```

2 En ejecución (Runtime)

Tipado Dinámico Fuerte: Sin conversión implícita de tipos.

Pattern Matching: Lanza `MatchError` si no coincide el patrón.

```
# Errores en tiempo de ejecución
10 + "5" # ➔ (ArithmeticError)
{:ok, x} = {:error, "Fallo"} # ➔ (MatchError)
```

CONCURRENCIA EN LA BEAM VM

Modelo de Actores: Procesos aislados extremadamente livianos sin memoria compartida.

Tolerancia a fallos ("Let it crash"): Si un proceso falla en tiempo de ejecución, un Supervisión/GenServer lo reinicia de inmediato sin afectar al resto del sistema.

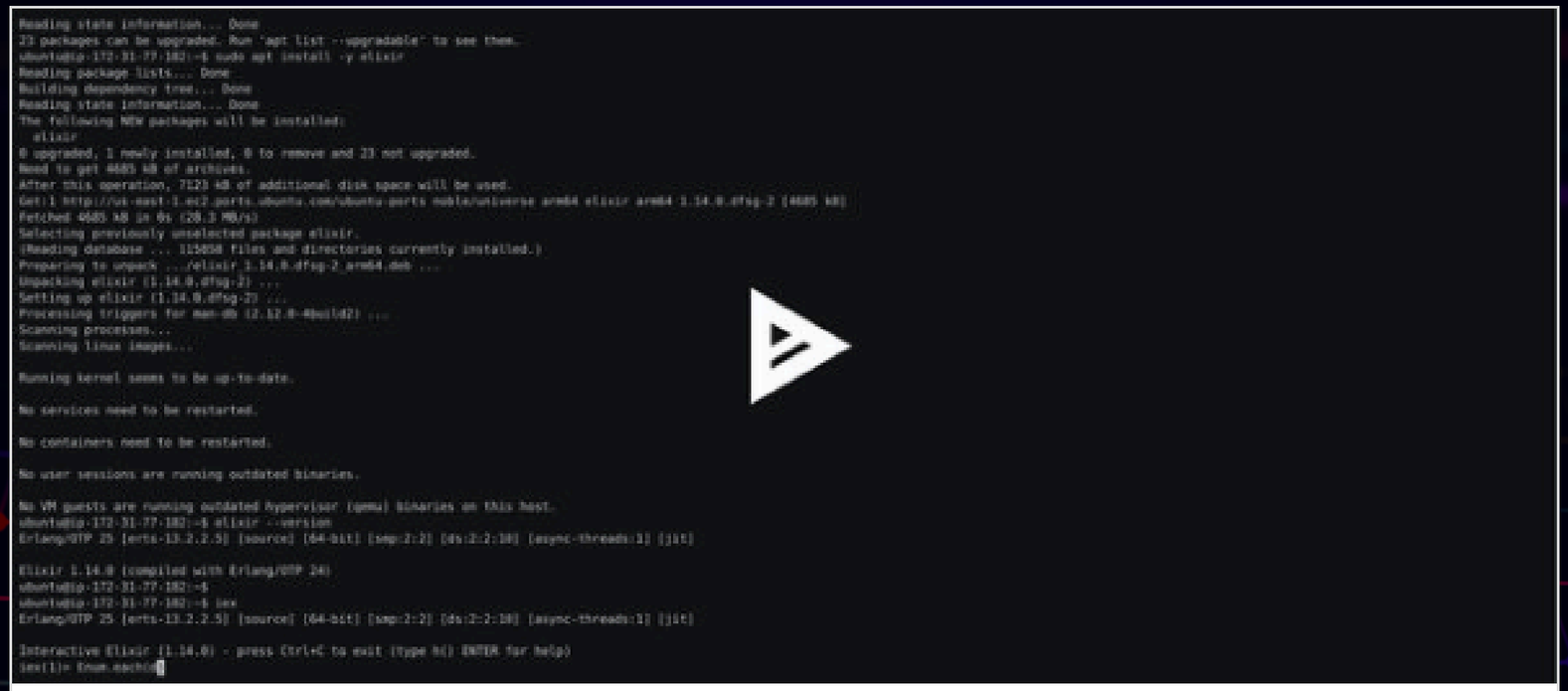
Paso de Mensajes: Comunicación asíncrona mediante direcciones de proceso (PID).

ejemplo_runtime.ex BEAM Process

```elixir
# 1. Crear proceso aislado con spawn
pid = spawn(fn ->
  receive do
    {:hola, nombre} ->
      IO.puts("¡Hola, \#{nombre}!")
  end
end)

# 2. Enviar mensaje asíncrono
send(pid, {:hola, "BEAM"})
# ➜ Salida: ¡Hola, BEAM!
```

GRABACIÓN ASCIINEMA



Demo Elixir — Equipo 4 (Unidad 1, PLF)

Demostración en vivo de Elixir para la exposición de la Unidad 1 (Introducción a lenguajes funcionales). Este demo muestra el uso del REPL interactivo (iex) para probar código en tiempo real, sin...

asciinema.org /

CASO REAL: DISCORD Y DE ELIXIR

- Discord construyó su prototipo original en Elixir desde el día 1

- Problema real: al crecer a comunidades de +30,000 usuarios, el envío de mensajes entre procesos tardaba hasta 2.1 segundos en hora pico

- Solución: librerías propias (Manifold, FastGlobal, Semaphore) aprovechando el modelo de concurrencia de la BEAM

- Resultado (2017): ~5,000,000 usuarios concurrentes, millones de eventos/segundo

- Conclusión de Discord: "si tuviéramos que empezar de nuevo, elegiríamos el mismo camino"

FUENTE IEEE

[1] S. Vishnevskiy, "How Discord Scaled Elixir to 5,000,000 Concurrent Users," Discord Engineering Blog, Jul. 6, 2017. [En línea]. Disponible: https://discord.com/blog/how-discord-scaled-elixir- to-5-000-000-concurrent-users

FUENTES BIBLIOGRAFICAS

[1] S. VISHNEVSKIY, "HOW DISCORD SCALED ELIXIR TO 5,000,000 CONCURRENT USERS," DISCORD ENGINEERING BLOG, JUL. 6, 2017. [ONLINE]. AVAILABLE: HTTPS://DISCORD.COM/BLOG/HOW-DISCORD-SCALED-ELIXIR-TO-5-000-000-CONCURRENT-USERS

[2] ELIXIR TEAM, "THE ELIXIR PROGRAMMING LANGUAGE," ELIXIR. [ONLINE]. AVAILABLE: HTTPS://ELIXIR-LANG.ORG/. [ACCESSED: SEP. 22, 2026].

[3] ERLANG/OTP TEAM, "OVERVIEW—OTP DESIGN PRINCIPLES," ERLANG DOCUMENTATION. [ONLINE]. AVAILABLE: HTTPS://WWW.ERLANG.ORG/DOC/SYSTEM/DESIGN_PRINCIPLES.HTML. [ACCESSED: SEP. 22, 2026].

[4] ERLANG/OTP TEAM, "GEN_SERVER—OTP DESIGN PRINCIPLES," ERLANG DOCUMENTATION. [ONLINE]. AVAILABLE: HTTPS://WWW.ERLANG.ORG/DOC/APPS/STDLIB/GEN_SERVER.HTML. [ACCESSED: SEP. 22, 2026].

[5] ERLANG/OTP TEAM, "SUPERVISOR—OTP DESIGN PRINCIPLES," ERLANG DOCUMENTATION. [ONLINE]. AVAILABLE: HTTPS://WWW.ERLANG.ORG/DOC/APPS/STDLIB/SUPERVISOR.HTML. [ACCESSED: SEP. 22, 2026].

[6] ELIXIR TEAM, "KERNEL.SPECIALFORMS," HEXDOCS. [ONLINE]. AVAILABLE: HTTPS://HEXDOCS.PM/ELIXIR/KERNEL.SPECIALFORMS.HTML. [ACCESSED: SEP. 22, 2026].

[7] ELIXIR TEAM, "GENSERVER," HEXDOCS. [ONLINE]. AVAILABLE: HTTPS://HEXDOCS.PM/ELIXIR/GENSERVER.HTML. [ACCESSED: SEP. 22, 2026].

[8] ELIXIR TEAM, "ENUMERABLE AND STREAMS," ELIXIR DOCUMENTATION. [ONLINE]. AVAILABLE: HTTPS://ELIXIR-LANG.ORG/GETTING-STARTED/ENUMERABLES-AND-STREAMS.HTML. [ACCESSED: SEP. 22, 2026].

GRACIAS POR SU ATENCIÓN